`k8s-03` tasks

1.Create a ClusterIP service for an Apache web server pod.

Objective

The objective of this task is to create an Apache web server pod and expose it internally using a Kubernetes ClusterIP service.

The ClusterIP service allows other pods within the Kubernetes cluster to communicate with the Apache web server using a stable internal IP address or service name.

2. Environment Details

Component	Details
Operating System	Windows 11
Container Platform	Docker Desktop
Kubernetes Tool	Minikube
Kubernetes Version	v1.37.0
Cluster Driver	Docker
Namespace	default
Apache Image	httpd
Service Type	ClusterIP

3. Verify Kubernetes Cluster

Before creating the Apache pod, verify that Minikube and Kubernetes are running.

Command:

```
None
minikube status
kubectl get nodes
```

Output:

```
None
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

NAME       STATUS   ROLES           AGE     VERSION
minikube   Ready    control-plane   3d23h   v1.37.0
```

Explanation:

- Minikube cluster is running successfully.
- Kubernetes API server is running.
- The Minikube node is in Ready status.
- The cluster is ready to create Kubernetes resources.

4. Create an Apache Web Server Pod

Create an Apache pod using the official `httpd` container image.

Command:

```
None
kubectl run apache-pod --image=httpd --port=80
```

Output:

```
None
pod/apache-pod created
```

Explanation:

Command option	Description
`kubectl run`	Creates a Kubernetes pod.
`apache-pod`	Name of the pod.
`--image=httpd`	Uses the Apache HTTP server image.
`--port=80`	Specifies the container port.

5. Verify the Apache Pod

Check whether the Apache pod is running successfully.

Command:

```
None
kubectl get pods -o wide
```

Output:

```
None
NAME         READY   STATUS    RESTARTS   AGE   IP
apache-pod   1/1     Running   0          12s   10.244.0.11
```

Explanation:

- `1/1`: Container is ready.
- `Running`: Apache pod is running successfully.
- `0`: No container restarts.
- `10.244.0.11`: Internal IP address assigned to the Apache pod.

6. Create a ClusterIP Service

Expose the Apache pod using a ClusterIP service.

Command:

```
None
kubectl expose pod apache-pod --name=apache-service --port=80
--target-port=80 --type=ClusterIP
```

Output:

```
None
service/apache-service exposed
```

Explanation:

Parameter	Description
`expose pod apache-pod`	Exposes the Apache pod.

`--name=apache-service`	Assigns a name to the service.
`--port=80`	Service port.
`--target-port=80`	Container port where Apache listens.
`--type=ClusterIP`	Creates an internal Kubernetes service.

7. Verify the ClusterIP Service

Check whether the service has been created successfully.

Command:

```
None
kubectl get services
```

Output:

```
None
NAME              TYPE        CLUSTER-IP       EXTERNAL-IP
PORT(S)
apache-service    ClusterIP   10.100.30.107    <none>        80/TCP
kubernetes        ClusterIP   10.96.0.1        <none>
443/TCP
nginx-clusterip   ClusterIP   10.102.221.240   <none>        80/TCP
nginx-nodeport    NodePort    10.99.120.232    <none>
80:30380/TCP
nginx-service     ClusterIP   10.109.206.157   <none>        80/TCP
```

Explanation:

The `apache-service` has been created with:

- Service type: ClusterIP
- Cluster IP: `10.100.30.107`
- Port: `80`
- External IP: None

ClusterIP is accessible internally within the Kubernetes cluster.

8. Describe the Apache Service

Use the following command to view detailed service information.

Command:

```
None
kubectl describe service apache-service
```

Important Output:

```
None
Name:         apache-service
Namespace:    default
Selector:     run=apache-pod
Type:         ClusterIP
IP:           10.100.30.107
Port:         80/TCP
TargetPort:   80/TCP
Endpoints:    10.244.0.11:80
```

Explanation:

- Selector: `run=apache-pod` identifies the Apache pod.
- ClusterIP: `10.100.30.107` is the internal service IP.
- TargetPort: `80` forwards traffic to Apache.
- Endpoints: `10.244.0.11:80` represents the Apache pod receiving service traffic.

9. Test Apache Web Server from Another Pod

Create a temporary BusyBox client pod and access Apache through the ClusterIP service name.

Command:

```
None
kubectl run apache-client --image=busybox:1.36 --restart=Never --rm -it -- wget -qO- http://apache-service
```

Output:

```
None
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
```

Explanation:

- BusyBox acts as a client pod.
- It sends an HTTP request to `apache-service`.
- Kubernetes resolves the service name internally.
- The service forwards the request to the Apache pod.
- Apache returns its default HTML web page.
- The temporary client pod is automatically deleted because of the `--rm` option.

The warning about falling back to streaming logs did not prevent the successful HTTP response.

10. Architecture

Apache Client Pod

BusyBox · Sends HTTP request

Apache ClusterIP Service

10.100.30.107

Port 80

Apache Pod

10.244.0.11:80 · httpd container

11. Important Kubernetes Concepts Learned

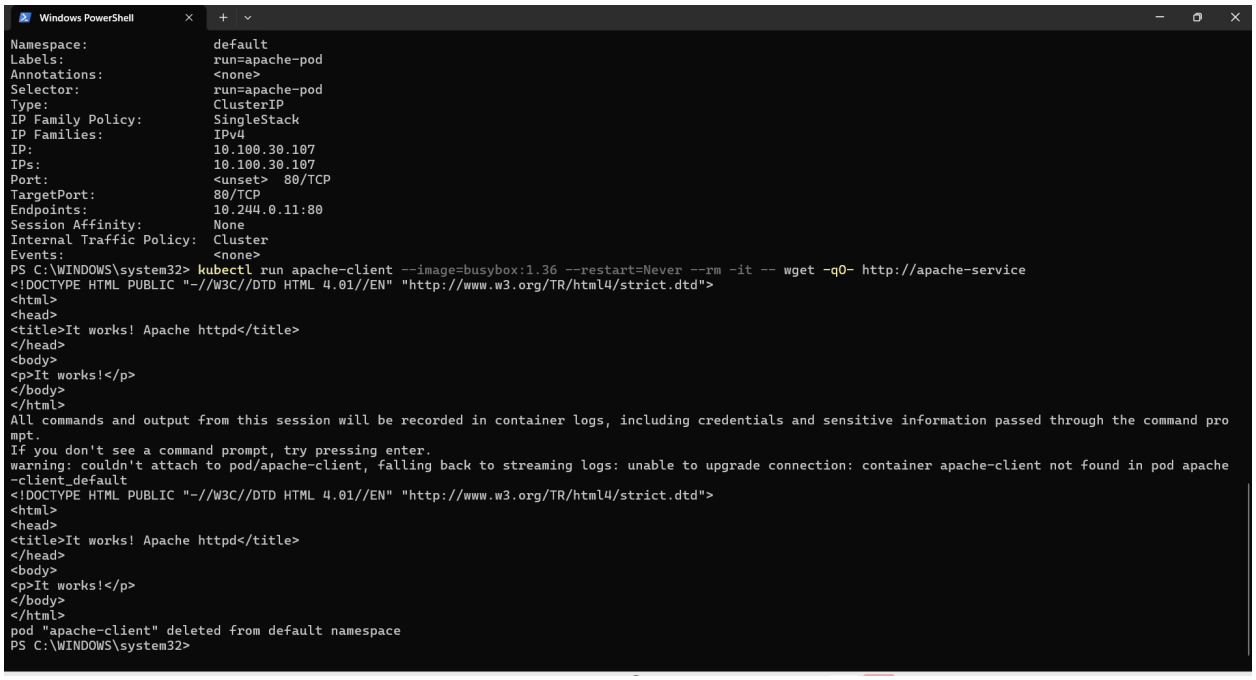
Concept	Explanation
Pod	Smallest deployable unit in Kubernetes.
Apache	Web server used to serve web pages.
ClusterIP	Default service type in Kubernetes.
Service	Provides a stable IP and DNS name to access pods.
Selector	Matches pod labels to connect services with pods.
Endpoint	IP address and port of the backend pod.
TargetPort	Container port receiving service traffic.

12. Conclusion

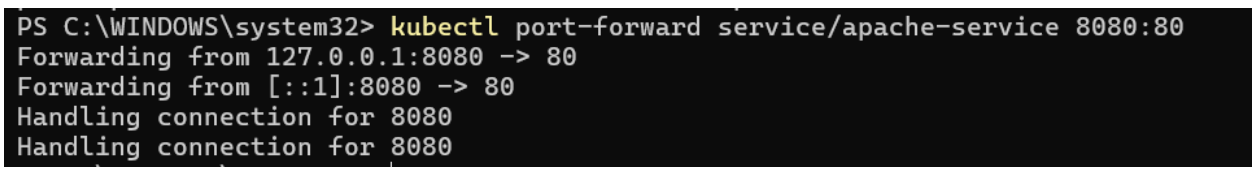
Successfully created an Apache web server pod using the `httpd` image and exposed it through a ClusterIP service.

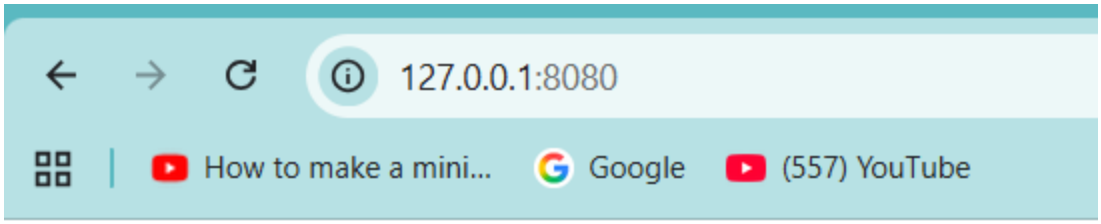
Verified the service using `kubectl get services` and `kubectl describe service`. Created a temporary BusyBox client pod and successfully accessed the Apache web server using the service name.

The successful HTTP response confirmed that internal communication between Kubernetes pods through a ClusterIP service is working correctly.



If see in browse you need expose port





It works!

K8s-03 – Task 1: Create a ClusterIP Service for an Apache Web Server Pod (YAML Method)

1. Objective

The objective of this task is to create an Apache web server Pod and expose it internally within the Kubernetes cluster using a ClusterIP Service, using a YAML configuration file.

We will use `kubectl apply -f` to create and manage Kubernetes resources declaratively.

2. What is ClusterIP?

ClusterIP is the default Service type in Kubernetes.

- It provides an internal IP address to access applications.
- It allows communication between Pods inside the Kubernetes cluster.
- It is not directly accessible from outside the cluster.
- It forwards traffic to Pods based on labels and selectors.

3. Environment Details

Component	Details

Operating System	Windows 11
Kubernetes Tool	Minikube
Container Runtime	Docker
Container Image	httpd (Apache)
Service Type	ClusterIP
Namespace	default

4. Create YAML Configuration File

Open PowerShell and create the YAML file:

```
None
notepad C:\k8s-03\apache-clusterip.yaml
```

Add the following configuration:

```
None
apiVersion: v1

kind: Pod

metadata:

  name: apache-pod

  labels:
```

```
    run: apache-pod
spec:
  containers:
    - name: apache-container
      image: httpd
      ports:
        - containerPort: 80
          protocol: TCP


---
apiVersion: v1
kind: Service
metadata:
  name: apache-service
spec:
  type: ClusterIP
  selector:
    run: apache-pod
  ports:
    - port: 80
      targetPort: 80
```

```
    protocol: TCP
```

Save and close the file.

5. Understand the YAML Configuration

The YAML file contains two Kubernetes resources: a Pod and a Service.

Pod configuration:

Field	Description
`apiVersion: v1`	Kubernetes API version
`kind: Pod`	Creates a Pod
`metadata.name`	Pod name is apache-pod
`labels`	Assigns a label to the Pod
`containers`	Defines the container
`image: httpd`	Uses Apache HTTP Server image
`containerPort: 80`	Apache listens on port 80

Service configuration:

Field	Description
`kind: Service`	Creates a Kubernetes Service
`name: apache-service`	Service name
`type: ClusterIP`	Internal Service
`selector`	Finds Pods with matching labels
`port: 80`	Service port
`targetPort: 80`	Container destination port

Important: The Pod label `run: apache-pod` must match the Service selector `run: apache-pod`. This is how the Service identifies the Apache Pod.

6. Apply the YAML Configuration

Execute:

```
None
kubectl apply -f C:\k8s-03\apache-clusterip.yaml
```

Expected output:

```
None
pod/apache-pod created

service/apache-service created
```

If the resources already exist and match the YAML, Kubernetes may display `unchanged` instead of `created`.

7. Verify the Apache Pod

Execute:

```
None
kubectl get pods -o wide
```

Expected result:

```
None
NAME         READY   STATUS    RESTARTS

apache-pod   1/1     Running   0
```

The Pod should be in the Running state.

8. Verify the ClusterIP Service

Execute:

```
None
kubectl get services
```

Your output showed:

```
None
NAME             TYPE        CLUSTER-IP      PORT(S)

apache-service   ClusterIP   10.100.30.107   80/TCP
```

The ClusterIP address is assigned automatically by Kubernetes.

9. Describe the Service

Execute:

```
None
kubectl describe service apache-service
```

Important details from your output:

```
None
Type:       ClusterIP

IP:         10.100.30.107

Port:       80/TCP

TargetPort: 80/TCP

Selector:   run=apache-pod

Endpoints:  10.244.0.14:80
```

What is an Endpoint?

An Endpoint is the IP address and port of a Pod selected by the Service.

Here, `apache-service` forwards traffic to the Apache Pod at `10.244.0.14:80`.

10. Test Apache Service Using BusyBox

Create a temporary BusyBox client Pod and access Apache through its Service name.

Execute:

```
None
kubectl run apache-client --image=busybox:1.36 --restart=Never --rm
-it -- wget -qO- http://apache-service
```

Your actual output:

```
None
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN">

<html>

<head>

<title>It works! Apache httpd</title>

</head>

<body>

<p>It works!</p>

</body>

</html>
```

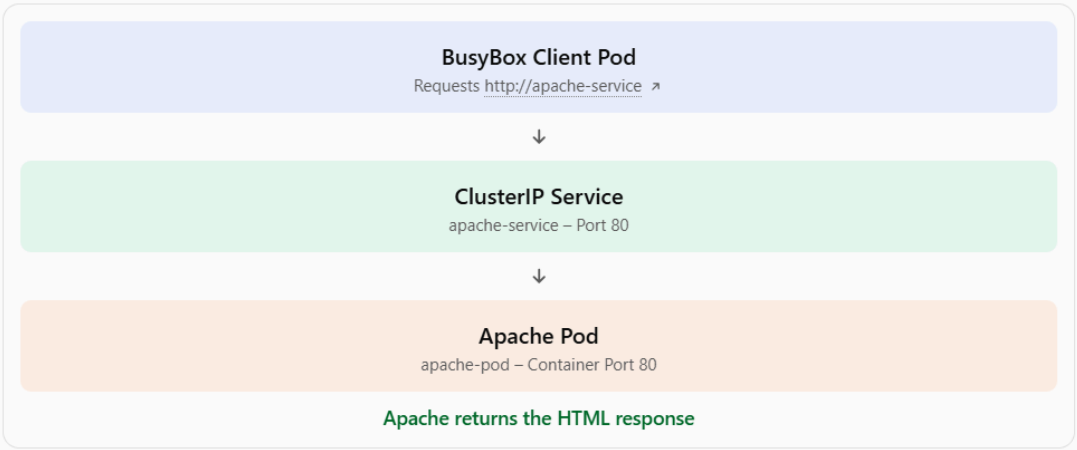
Connectivity successful

This confirms that the BusyBox client successfully communicated with the Apache Pod through the ClusterIP Service.

The temporary `apache-client` Pod was automatically deleted after the test.

Note: You also saw a warning about attaching to the short-lived client Pod. It is harmless in this case because the Apache HTML response was successfully returned.

11. Architecture / Working Flow



12. Important Commands Used

Command	Purpose
`kubectl apply -f file.yaml`	Create or update resources using YAML
`kubectl get pods -o wide`	Display Pod details and IP
`kubectl get services`	Display Services
`kubectl describe service apache-service`	Display Service configuration and endpoints
`kubectl get endpoints apache-service`	Display selected Pod endpoints

`kubectl delete -f file.yaml`	Delete resources defined in YAML

13. Concepts Learned

1. How to create a Pod using YAML.
2. How to create a ClusterIP Service using YAML.
3. How labels and selectors connect Pods with Services.
4. How Kubernetes assigns a ClusterIP address.
5. How Services forward traffic to Pod endpoints.
6. How to test internal communication using a BusyBox client.
7. How to use declarative resource management with `kubectl apply`.

14. Conclusion

In this task, we successfully created an Apache web server Pod and exposed it internally using a ClusterIP Service through a YAML configuration file.

We verified the Pod, Service, and Endpoint and successfully tested communication using a temporary BusyBox client Pod.

Result: Task 1 – Create a ClusterIP Service for an Apache web server Pod using YAML – completed successfully.

2.Expose an Nginx pod externally using a NodePort service.

K8s-03 – Task 2: Expose an Nginx Pod Externally Using a NodePort Service (YAML Method)

1. Objective

The objective of this task is to expose an Nginx web server outside the Kubernetes cluster using a NodePort Service.

We will create a NodePort Service using a YAML configuration file and access the Nginx application through a browser.

2. What is NodePort?

NodePort is a Kubernetes Service type that exposes an application on a port of each Kubernetes node.

- It allows users to access applications from outside the cluster.
- It uses a port number, usually in the range `30000–32767`.
- It forwards incoming traffic to the application Pods.
- It works with the ClusterIP Service internally.

Example: If the NodePort is `30080`, users can access the application using:

```
None
http://<Node-IP>:30080
```

3. Environment Details

Component	Details
Operating System	Windows 11
Kubernetes Tool	Minikube
Container Runtime	Docker
Container Image	nginx

Service Type	NodePort
Namespace	default

4. Verify the Nginx Pod

Before creating the Service, check whether the Nginx Pod is running.

Execute:

```
None
kubectl get pods -o wide --show-labels
```

Check that the Nginx Pod is in the `Running` state.

Important: The Service selector must match the Nginx Pod label. For example, if the Pod has the label `run=nginx`, use the same selector in the Service YAML.

5. Create YAML Configuration File

Create a YAML file using Notepad:

```
None
notepad C:\k8s-03\nginx-nodeport.yaml
```

Add the following configuration:

```
None
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport-task2
spec:
```

```
  type: NodePort
  selector:
    run: nginx
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
```

Save and close the file.

This example assumes the Nginx Pod has the label run=nginx. Verify the label before applying it. Kubernetes will automatically assign an available NodePort because we have not specified one.

6. Understand the YAML Configuration

Field	Description
`apiVersion: v1`	Kubernetes API version
`kind: Service`	Creates a Service
`metadata.name`	Name of the Service
`type: NodePort`	Exposes the application externally

`selector`	Identifies the Nginx Pod
`port: 80`	Service port
`targetPort: 80`	Nginx container port
`protocol: TCP`	Communication protocol

7. Apply the YAML Configuration

Execute:

```
None
kubectl apply -f C:\k8s-03\nginx-nodeport.yaml
```

Expected output for a newly created Service:

```
None
service/nginx-nodeport-task2 created
```

If it already exists, Kubernetes may display:

```
None
service/nginx-nodeport-task2 unchanged
```

8. Verify the NodePort Service

Execute:

```
None
kubectl get services
```

Example output:

```
None
NAME                    TYPE       CLUSTER-IP       PORT(S)
nginx-nodeport-task2    NodePort   10.106.153.131
80:32646/TCP
```

Your earlier output showed `nginx-nodeport-task2` with NodePort `32646`.

Understanding `80:32646/TCP`:

- `80` – Service port.
- `32646` – NodePort accessible through the Minikube node.
- `TCP` – Communication protocol.

9. Describe the Service

Execute:

```
None
kubectl describe service nginx-nodeport-task2
```

Check these important details:

- Type: NodePort
- Selector: `run=nginx`
- Port: 80
- TargetPort: 80
- NodePort: assigned port
- Endpoints: Nginx Pod IP and port 80

The Endpoints should contain an Nginx Pod IP. If they show `<none>`, check whether the Service selector matches the Pod labels.

10. Access Nginx Through Minikube

Execute:

```
None
      minikube service nginx-nodeport-task2 --url
```

Minikube will return a URL. Copy that URL and open it in Chrome.

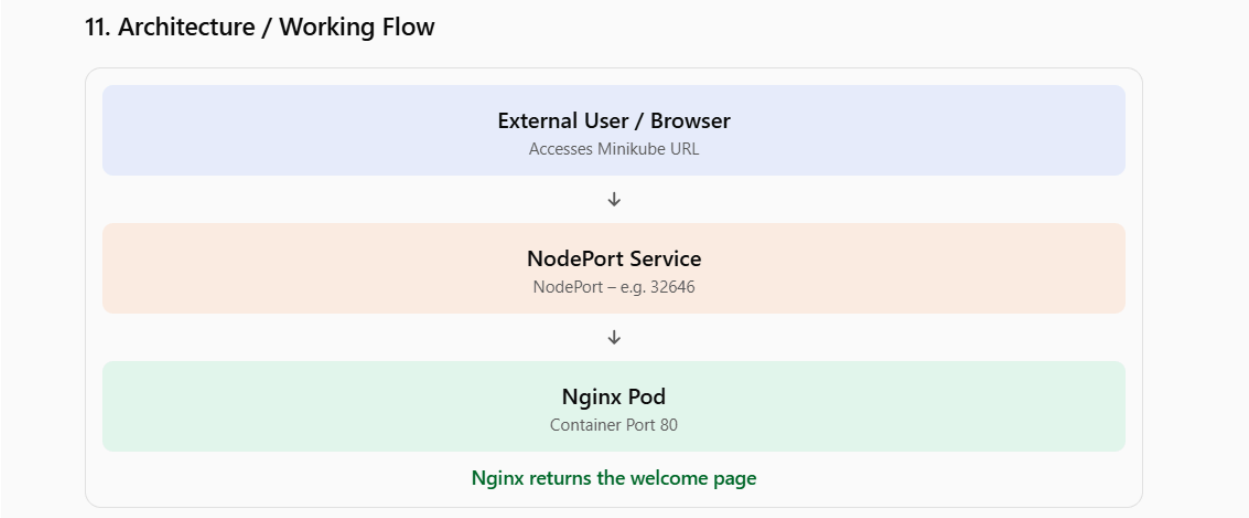
Expected browser result:

> How to deploy Nginx on Kubernetes using Helm Chart
> theairtips.com

You should see:

Welcome to nginx!

This confirms that the Nginx application is accessible through the NodePort Service.



12. Important Commands Used

Command	Purpose
`kubectl get pods -o wide --show-labels`	Check Pod status and labels

`kubectl apply -f nginx-nodeport.yaml`	Create Service using YAML
`kubectl get services`	Verify Services
`kubectl describe service nginx-nodeport-task2`	Check Service details
`kubectl get endpoints nginx-nodeport-task2`	Check backend Pod endpoints
`minikube service nginx-nodeport-task2 --url`	Get browser access URL
`kubectl delete -f nginx-nodeport.yaml`	Delete the Service defined in YAML

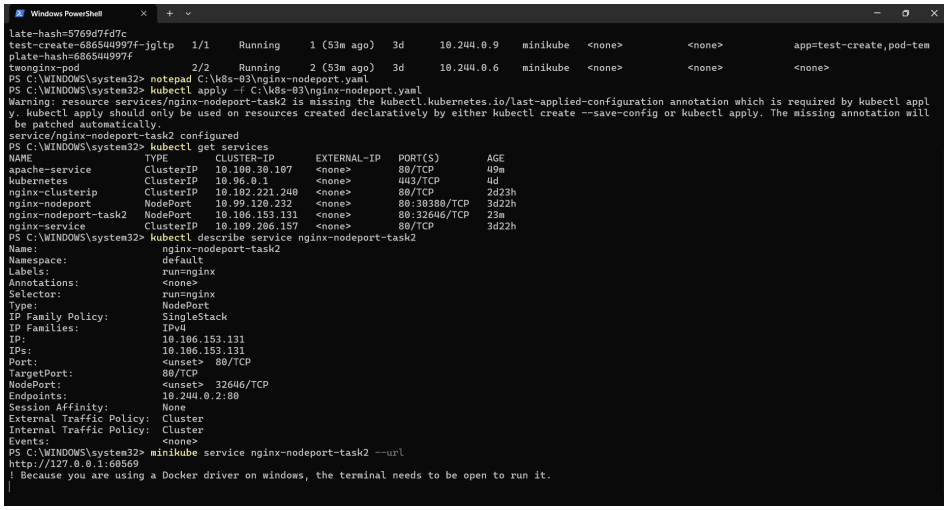
13. Concepts Learned

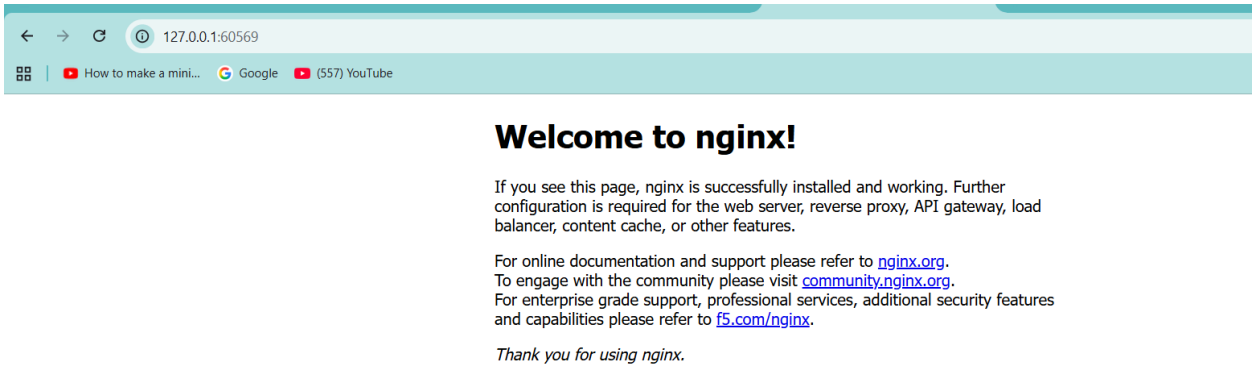
1. What is a NodePort Service?
2. Difference between ClusterIP and NodePort.
3. How to expose an application outside the Kubernetes cluster.
4. How labels and selectors connect Services to Pods.
5. How to access an Nginx application using a browser.
6. How to create Kubernetes Services using YAML.

14. Conclusion

In this task, we created a NodePort Service to expose an Nginx application outside the Kubernetes cluster.

We verified the Service configuration, checked its endpoints, and accessed the Nginx application through a browser using the Minikube Service URL.





3.Deploy a ReplicationController to maintain 3 replicas of an Nginx pod.

K8s-03 – Task 3: Deploy a ReplicationController to Maintain 3 Replicas of an Nginx Pod (YAML Method)

1. Objective

The objective of this task is to deploy a Kubernetes ReplicationController that maintains three replicas of an Nginx web server Pod.

If any Pod fails or gets deleted, the ReplicationController automatically creates a replacement Pod to maintain the desired number of replicas.

2. What is a ReplicationController?

A ReplicationController is a Kubernetes resource that ensures a specified number of Pod replicas are running at all times.

Main functions:

- Maintains the desired number of Pods.
- Automatically creates replacement Pods if existing Pods are deleted.
- Helps maintain application availability.
- Supports scaling applications by increasing or decreasing replicas.

3. Environment Details

Component	Details
Cloud Provider	AWS
Compute Service	EC2
Operating System	Ubuntu 24.04.5 LTS
Kubernetes Version	v1.37.2
Container Runtime	containerd 2.2.1

Master Node	k8s-master
Worker Nodes	k8s-worker-1, k8s-worker-2
Container Image	nginx
Resource	ReplicationController
Desired Replicas	3

4. Verify Kubernetes Cluster Nodes

Before creating the ReplicationController, verify that all Kubernetes nodes are running.

Execute:

```
None
kubectl get nodes -o wide
```

Your actual output:

```
None
NAME           STATUS   ROLES           VERSION
k8s-master     Ready    control-plane   v1.37.2
k8s-worker-1   Ready    <none>          v1.37.2
k8s-worker-2   Ready    <none>          v1.37.2
```

All three nodes were in the `Ready` state.

5. Create YAML Configuration File

Create a YAML file on the master node:

```
None
nano nginx-rc.yaml
```

Add the following configuration:

```
None
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginx-rc
  template:
    metadata:
      labels:
        app: nginx-rc
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Save and exit Nano:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

6. Understand the YAML Configuration

Field	Description
`apiVersion: v1`	Kubernetes API version
`kind: ReplicationController`	Creates a ReplicationController
`metadata.name`	Name of the ReplicationController
`replicas: 3`	Maintains three Pods
`selector`	Identifies Pods managed by the controller
`template`	Defines the Pod configuration
`labels`	Assigns a label to the Pods
`containers`	Defines the container
`image: nginx`	Uses the Nginx container image
`containerPort: 80`	Nginx application port

Important: The selector `app: nginx-rc` must match the Pod template label `app: nginx-rc`. This allows the ReplicationController to identify and manage its Pods.

7. Apply the YAML Configuration

Execute:

```
None
kubectl apply -f nginx-rc.yaml
```

Your actual output:

```
None
replicationcontroller/nginx-rc created
```

This confirms that the ReplicationController was successfully created.

8. Verify ReplicationController

Execute:

```
None
kubectl get rc
```

Your actual output:

```
None
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   3         3         3       18s
```

Understanding the output:

Column	Value	Meaning

DESIRED	3	Required number of Pods
CURRENT	3	Actual number of Pods
READY	3	Number of Pods ready to serve requests

All three replicas were successfully created.

9. Verify Nginx Pods

Execute:

```
None
kubectl get pods -o wide
```

Your actual output:

```
None
NAME                     READY   STATUS    IP           NODE
nginx-69b9cdbbdd-ckl4s   1/1     Running   10.244.1.4
k8s-worker-1
nginx-69b9cdbbdd-mb55d   1/1     Running   10.244.2.5
k8s-worker-2
nginx-rc-2pcmv           1/1     Running   10.244.2.6
k8s-worker-2
nginx-rc-4hkss           1/1     Running   10.244.1.5
k8s-worker-1
nginx-rc-lbv6z           1/1     Running   10.244.1.6
k8s-worker-1
```

Observation:

- Three Pods were created by the new ReplicationController.
- Two additional Nginx Pods belonged to an existing Deployment.
- The three new Pods were distributed across both worker nodes.
- All five Pods were running successfully.

10. Test Automatic Pod Replacement

One of the main features of a ReplicationController is automatic Pod replacement.

First, identify the Pods:

```
None
kubectl get pods
```

Delete one Pod managed by the ReplicationController:

```
None
kubectl delete pod nginx-rc-2pcmv
```

Your actual output:

```
None
pod "nginx-rc-2pcmv" deleted from default namespace
```

11. Verify Automatic Pod Replacement

After deleting the Pod, execute:

```
None
kubectl get pods -o wide
```

Your actual output showed:

```
None
NAME                    READY   STATUS    IP          NODE
```

```
nginx-69b9cdbbdd-ckl4s   1/1     Running   10.244.1.4
k8s-worker-1
nginx-69b9cdbbdd-mb55d   1/1     Running   10.244.2.5
k8s-worker-2
nginx-rc-4hkss           1/1     Running   10.244.1.5
k8s-worker-1
nginx-rc-lbv6z           1/1     Running   10.244.1.6
k8s-worker-1
nginx-rc-rvqmg           1/1     Running   10.244.2.7
k8s-worker-2
```

Important observation:

The deleted Pod `nginx-rc-2pcmv` was replaced by a new Pod named `nginx-rc-rvqmg`.

The replacement Pod received a new IP address: `10.244.2.7`.

This proves that the ReplicationController automatically created a replacement Pod.

12. Final Verification

Execute:

```
None
kubectl get rc
```

Your actual output:

```
None
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   3         3         3       3m55s
```

All three replicas were maintained successfully.

Task 3 completed successfully

13. Architecture / Working Flow



14. Important Commands Used

Command	Purpose
`kubectl get nodes -o wide`	Verify Kubernetes nodes
`nano nginx-rc.yaml`	Create YAML configuration
`cat -n nginx-rc.yaml`	Display YAML with line numbers
`kubectl apply -f nginx-rc.yaml`	Create ReplicationController
`kubectl get rc`	Verify ReplicationController
`kubectl get pods -o wide`	Display Pod details

`kubectl delete pod <pod-name>`	Delete a specific Pod
`kubectl get rc`	Confirm desired replica count

15. Concepts Learned

1. What is a ReplicationController?
2. How to create a ReplicationController using YAML.
3. How to configure three Nginx replicas.
4. How selectors and labels work.
5. How Kubernetes distributes Pods across worker nodes.
6. How a ReplicationController detects a deleted Pod.
7. How Kubernetes automatically creates replacement Pods.
8. How to verify desired, current, and ready replica counts.

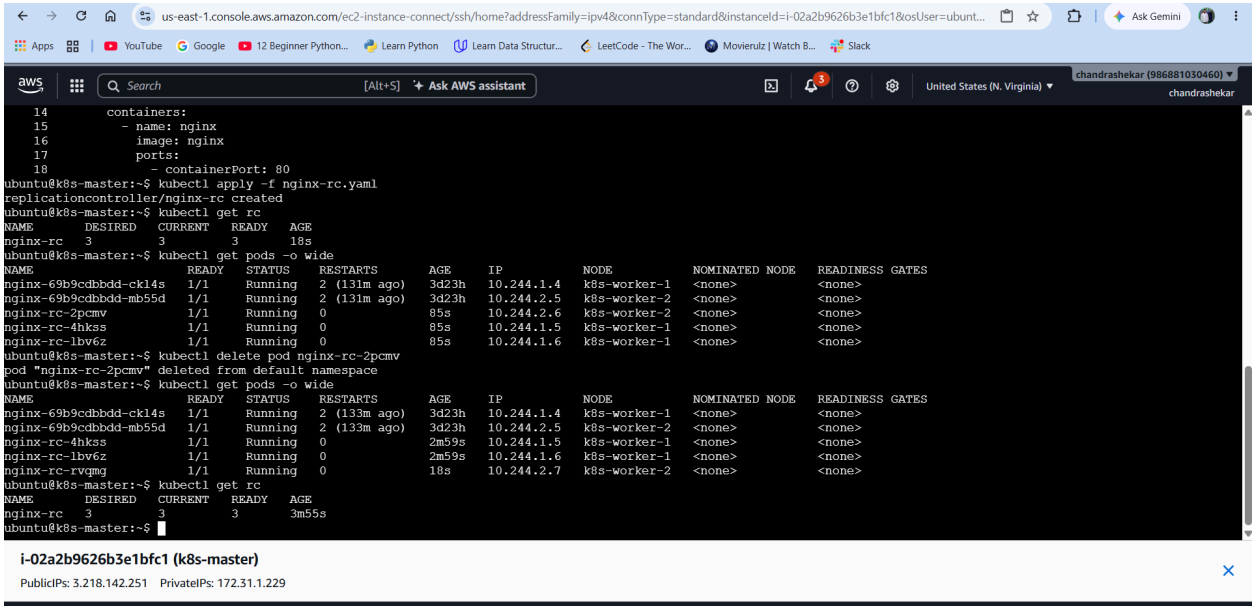
16. Conclusion

In this task, we successfully deployed a ReplicationController on an AWS EC2 Kubernetes cluster with one master node and two worker nodes.

We configured the ReplicationController to maintain three Nginx replicas using a YAML file. We verified that all three Pods were running successfully across the worker nodes.

We also deleted one Pod manually and observed that Kubernetes automatically created a replacement Pod to maintain the desired replica count.

Final Result: Task 3 – Deploy a ReplicationController to maintain three replicas of an Nginx Pod – completed successfully.



4.Scale the ReplicationController from 3 replicas to 5 replicas.

K8s-03 – Task 4: Scale the ReplicationController from 3 Replicas to 5 Replicas

Completed successfully

1. Objective

The objective of this task is to scale an existing Nginx ReplicationController from 3 replicas to 5 replicas using the `kubectl scale` command.

This task demonstrates how Kubernetes automatically creates additional Pods when we increase the desired replica count.

2. Environment details

Component	Details
Platform	AWS EC2
Operating System	Ubuntu 24.04.5 LTS

Kubernetes version	v1.37.2
Container Runtime	containerd
Master node	k8s-master
Worker nodes	k8s-worker-1, k8s-worker-2
Resource type	ReplicationController
ReplicationController name	nginx-rc
Container image	nginx

3. What is scaling in Kubernetes?

Scaling means increasing or decreasing the number of Pod replicas managed by a Kubernetes controller.

For example:

- Before scaling: 3 Nginx Pods.
- After scaling: 5 Nginx Pods.
- Kubernetes automatically creates or deletes Pods to match the desired replica count.

A ReplicationController continuously monitors the number of Pods and maintains the required number.

4. Verify the existing ReplicationController

Before scaling, check the current ReplicationController:

Command:

```
None
kubectl get rc
```

Output:

```
None
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   3         3         3       10m
```

Explanation:

Column	Meaning	Value
DESIRED	Number of Pods requested	3
CURRENT	Number of Pods currently created	3
READY	Number of Pods ready to serve requests	3

The output confirms that the ReplicationController is currently maintaining three Pods.

5. Verify the existing Pods

To display only the Pods managed by `nginx-rc`, execute:

```
None
kubectl get pods -l app=nginx-rc -o wide
```

Output:

```
None
NAME              READY   STATUS    RESTARTS   AGE
nginx-rc-4hkss    1/1     Running   0          10m
nginx-rc-lbv6z    1/1     Running   0          10m
nginx-rc-rvqmg    1/1     Running   0          8m2s
```

Explanation:

- `-l app=nginx-rc` filters Pods using the specified label.
- `-o wide` displays additional information, including Pod IP and worker node.
- All three Pods are in the Running state.

6. Scale the ReplicationController to 5 replicas

Execute the following command on the Kubernetes master node:

Command:

```
None
kubectl scale rc nginx-rc --replicas=5
```

Output:

```
None
replicationcontroller/nginx-rc scaled
```

Explanation:

Command part	Meaning
`kubectl`	Kubernetes command-line tool
`scale`	Changes the number of replicas

`rc`	Specifies ReplicationController
`nginx-rc`	Name of the ReplicationController
`--replicas=5`	Sets the desired number of Pods to five

Kubernetes compares the current number of Pods (3) with the desired number (5) and automatically creates two additional Pods.

7. Verify the ReplicationController after scaling

Execute:

```
None
kubectl get rc
```

Output:

```
None
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   5         5         5       11m
```

Explanation:

Column	Before scaling	After scaling
DESIRED	3	5

CURRENT	3	5
READY	3	5

The output confirms that the ReplicationController successfully maintains five ready Pods.

8. Verify all five Nginx Pods

Execute:

```
None
kubectl get pods -l app=nginx-rc -o wide
```

Output:

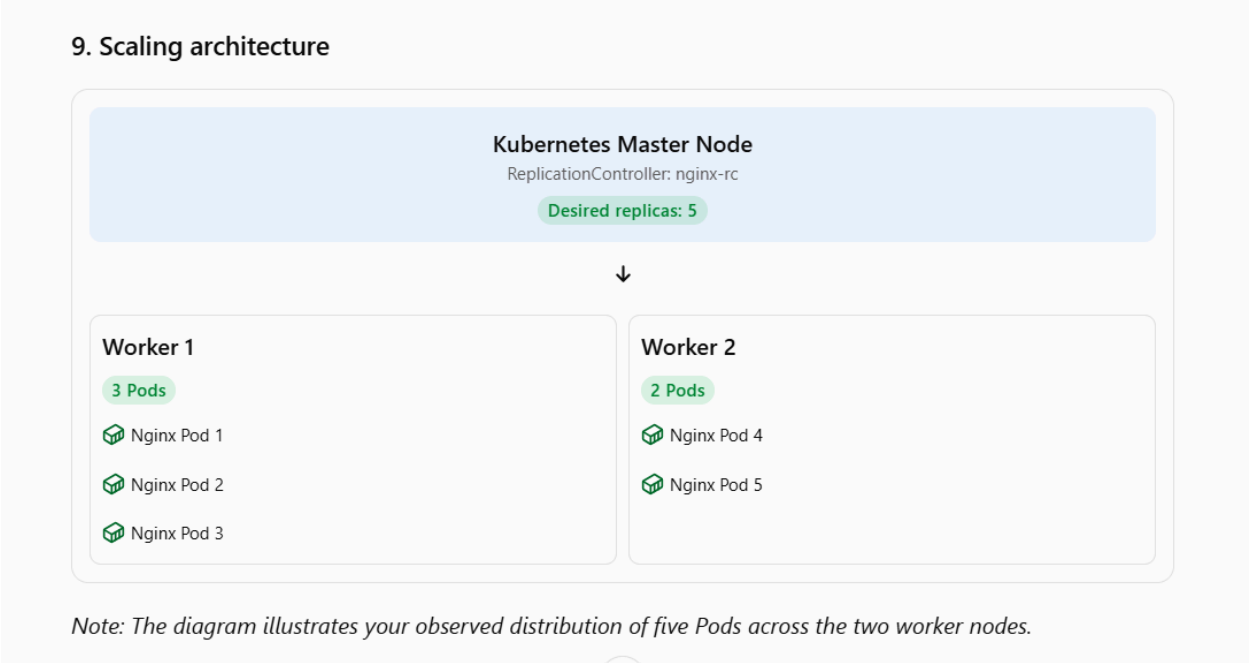
```
None
NAME             READY   STATUS    RESTARTS   AGE    IP
NODE
nginx-rc-4gxgc   1/1     Running   0          46s    10.244.1.7
k8s-worker-1
nginx-rc-4hkss   1/1     Running   0          11m    10.244.1.5
k8s-worker-1
nginx-rc-97jc9   1/1     Running   0          46s    10.244.2.8
k8s-worker-2
nginx-rc-lbv6z   1/1     Running   0          11m    10.244.1.6
k8s-worker-1
nginx-rc-rvqmg   1/1     Running   0          9m1s   10.244.2.7
k8s-worker-2
```

Explanation:

- Five Nginx Pods are running successfully.
- Three Pods are running on `k8s-worker-1`.
- Two Pods are running on `k8s-worker-2`.

- Each Pod has a separate IP address.
- All Pods show `1/1 Running`.

9. Scaling architecture



10. Important Kubernetes commands

Command	Purpose
`kubectl get rc`	List ReplicationControllers
`kubectl describe rc nginx-rc`	Display RC details
`kubectl get pods -l app=nginx-rc`	List Pods managed by the RC

`kubectl scale rc nginx-rc --replicas=5`	Scale to five Pods
`kubectl get pods -o wide`	Display Pod IPs and worker nodes

11. Important concepts learned

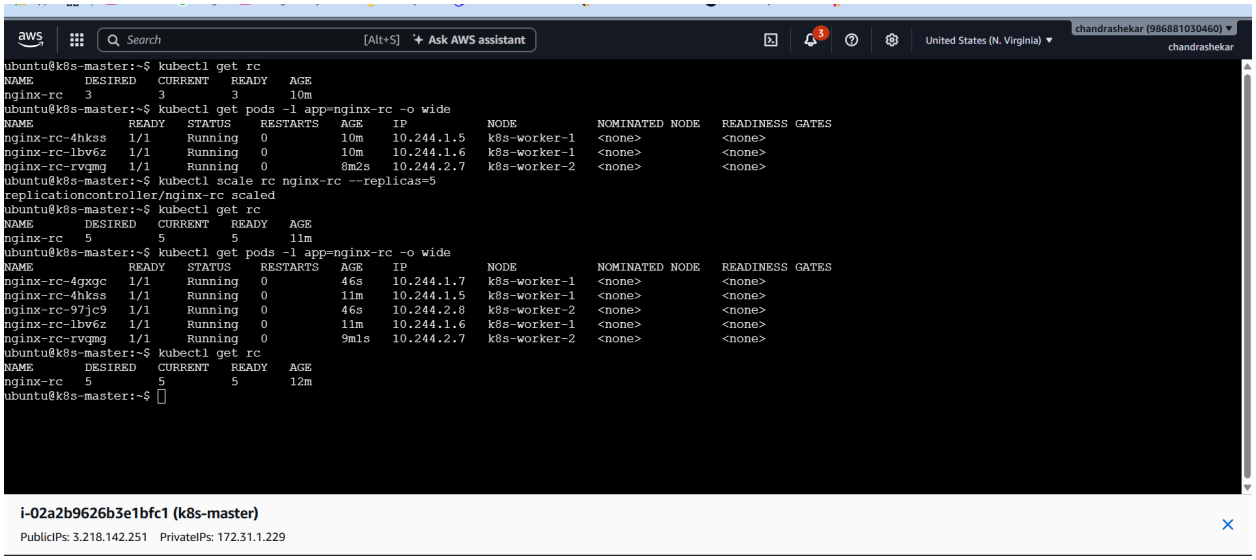
1. Scaling: Increasing or decreasing the number of Pod replicas.
2. Desired state: The number of replicas requested by the user.
3. Current state: The number of Pods currently created.
4. Automatic Pod creation: Kubernetes creates additional Pods when the replica count increases.
5. Self-healing: ReplicationController creates replacement Pods when managed Pods are deleted or fail.
6. Worker nodes: Kubernetes schedules Pods across available worker nodes.

12. Conclusion

In this task, we successfully scaled the Nginx ReplicationController from 3 replicas to 5 replicas using the `kubectl scale` command.

Initially, the ReplicationController maintained three Nginx Pods. After executing the scaling command, Kubernetes automatically created two additional Pods.

Finally, we verified that all five Pods were running successfully across the two worker nodes.



5.Create a ReplicaSet to manage pods based on multiple labels (prod and test).

K8s-03 – Task 5: Create a ReplicaSet to Manage Pods Based on Multiple Labels (prod and test)

Completed successfully

1. Objective

The objective of this task is to create a Kubernetes ReplicaSet that manages Nginx Pods based on multiple label values using the `matchExpressions` selector and the `In` operator.

In this task, we configure the ReplicaSet to match Pods with either of the following labels:

- `env: prod`
- `env: test`

We configure the ReplicaSet to maintain four Nginx Pod replicas.

2. What is a ReplicaSet?

A ReplicaSet is a Kubernetes resource that maintains a specified number of identical Pods.

Its main responsibilities are:

- Maintaining the desired number of Pods.
- Creating new Pods when the number decreases.

- Replacing failed or deleted Pods.
- Managing Pods based on labels and selectors.

Example: If we configure four replicas, Kubernetes ensures that four Pods are running and ready.

3. What are Labels and Selectors?

Labels are key-value pairs assigned to Kubernetes resources.

Example:

```
None
      labels:
        env: prod
```

Here:

- `env` is the label key.
- `prod` is the label value.

Selectors are used by Kubernetes controllers to identify resources based on their labels.

In this task, we use `matchExpressions` to select Pods based on multiple possible label values.

4. Environment details

Component	Details
Platform	AWS EC2
Operating System	Ubuntu 24.04.5 LTS
Kubernetes version	v1.37.2

Container Runtime	containerd
Master node	k8s-master
Worker node 1	k8s-worker-1
Worker node 2	k8s-worker-2
Resource type	ReplicaSet
ReplicaSet name	nginx-rs
Container image	nginx
Desired replicas	4

5. Verify Kubernetes cluster nodes

Before creating the ReplicaSet, verify that all cluster nodes are ready.

Command:

```
None
kubectl get nodes
```

Output:

```
None
NAME           STATUS   ROLES           AGE     VERSION
```

```
k8s-master     Ready    control-plane   3d23h   v1.37.2
k8s-worker-1   Ready    <none>          3d23h   v1.37.2
k8s-worker-2   Ready    <none>          3d23h   v1.37.2
```

Explanation:

All three Kubernetes nodes are in the `Ready` state.

6. Check existing ReplicaSets

Command:

```
None
kubectl get rs
```

Output:

```
None
NAME                DESIRED   CURRENT   READY   AGE
nginx-69b9cdbbdd    2         2         2       3d23h
```

An existing ReplicaSet was already present from the earlier Nginx Deployment task.

We created a new ReplicaSet named `nginx-rs` to perform this assignment without modifying the existing resource.

7. Create the ReplicaSet YAML file

Create a new YAML file using the Nano editor.

Command:

```
None
nano nginx-rs.yaml
```

Add the following configuration:

File name: `nginx-rs.yaml`

```
None
      apiVersion: apps/v1
      kind: ReplicaSet
      metadata:
        name: nginx-rs
      spec:
        replicas: 4
        selector:
          matchExpressions:
            - key: env
              operator: In
              values:
                - prod
                - test
        template:
          metadata:
            labels:
              env: prod
          spec:
            containers:
              - name: nginx
                image: nginx
                ports:
                  - containerPort: 80
```

Save the file:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

8. Explanation of YAML configuration

YAML field	Explanation
`apiVersion: apps/v1`	Specifies the Kubernetes API version
`kind: ReplicaSet`	Defines the resource type
`metadata.name`	Name of the ReplicaSet
`replicas: 4`	Maintains four Pods
`selector`	Defines which Pods the ReplicaSet manages
`matchExpressions`	Defines advanced label matching rules
`key: env`	Label key used for selection
`operator: In`	Matches any one of the specified values
`values: prod, test`	Allowed label values

`template`	Defines the Pod configuration
`labels.env: prod`	Assigns the `prod` label to created Pods
`image: nginx`	Uses the Nginx container image
`containerPort: 80`	Specifies the container port

9. Understanding the `In` operator

The most important part of this task is the `In` operator.

Configuration:

```
None
      matchExpressions:
        - key: env
          operator: In
          values:
            - prod
            - test
```

The `In` operator checks whether the label value belongs to the specified list.

Pod label	Result

`env=prod`	Matches
`env=test`	Matches
`env=dev`	Does not match

Important: This is an OR condition.

A Pod can match if its `env` label is either `prod` OR `test`.

Our Pod template assigns `env: prod`, so the four Pods created in this task all have the `prod` label.

10. Verify the YAML file

Command:

```
None
cat nginx-rs.yaml
```

The YAML configuration was displayed successfully and verified before applying it.

11. Apply the ReplicaSet configuration

Execute the following command to create the ReplicaSet.

Command:

```
None
kubectl apply -f nginx-rs.yaml
```

Output:

```
None
      replicaset.apps/nginx-rs created
```

Explanation:

The `kubectl apply` command reads the YAML file and creates the ReplicaSet in the Kubernetes cluster.

12. Verify the ReplicaSet

Command:

```
None
      kubectl get rs
```

Output:

```
None
      NAME                DESIRED   CURRENT   READY   AGE
      nginx-69b9cdbbdd    2         2         2       3d23h
      nginx-rs            4         4         4       24s
```

Explanation:

Column	Value	Meaning
DESIRED	4	Four Pods requested
CURRENT	4	Four Pods created
READY	4	All four Pods are ready

This confirms that the ReplicaSet was created successfully.

13. Verify the four Nginx Pods

To display only Pods having the `env=prod` label, execute:

Command:

```
None
kubectl get pods -l env=prod -o wide
```

Output:

```
None
NAME             READY   STATUS    RESTARTS   AGE   IP
NODE
nginx-rs-64nkt   1/1     Running   0          46s   10.244.2.9
k8s-worker-2
nginx-rs-lgp2h   1/1     Running   0          46s
10.244.2.10   k8s-worker-2
nginx-rs-px8xd   1/1     Running   0          46s   10.244.1.9
k8s-worker-1
nginx-rs-zmnrm   1/1     Running   0          46s   10.244.1.8
k8s-worker-1
```

Explanation:

- Four Nginx Pods were created.
- All four Pods are in `Running` status.
- Three? No: two Pods are running on `k8s-worker-1` and two on `k8s-worker-2`.
- Each Pod has a separate IP address.
- All Pods have the `env=prod` label.

14. Describe the ReplicaSet

To inspect the ReplicaSet configuration and selector, execute:

Command:

```
kubectl describe rs nginx-rs
```

Important output:

```
Name:         nginx-rs
Namespace:    default
Selector:     env in (prod,test)

Replicas:     4 current / 4 desired

Pods Status:  4 Running / 0 Waiting / 0 Succeeded / 0 Failed

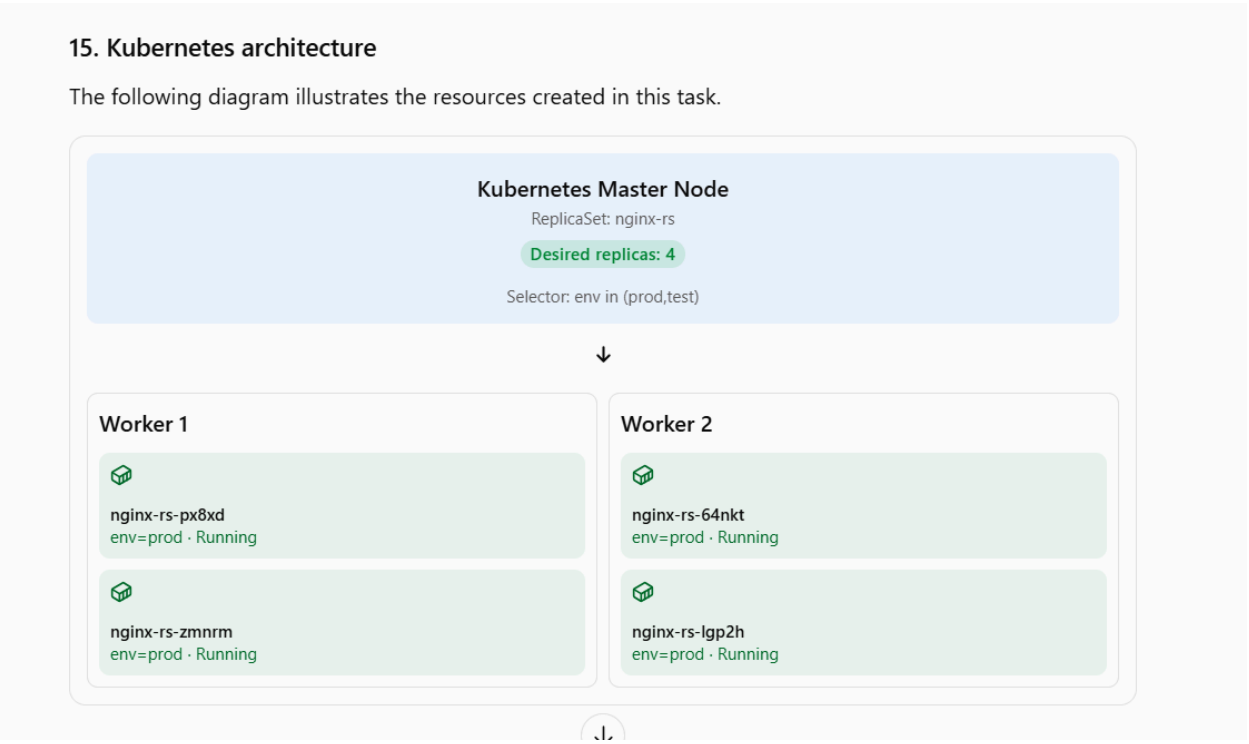
Pod Template:
  Labels:  env=prod
  Containers:
   nginx:
    Image:         nginx
    Port:          80/TCP
```

Explanation:

- `Selector: env in (prod,test)` confirms that the ReplicaSet accepts either label value.
- `Replicas: 4 current / 4 desired` confirms four Pods are maintained.
- `Pods Status: 4 Running` confirms all four Pods are running.
- `Labels: env=prod` confirms the label assigned through the Pod template.

15. Kubernetes architecture

The following diagram illustrates the resources created in this task.



16. Important Kubernetes commands

Command	Purpose
`kubectl get nodes`	Display cluster nodes
`kubectl get rs`	List ReplicaSets
`kubectl apply -f nginx-rs.yaml`	Create or update ReplicaSet
`kubectl get pods -l env=prod`	Display Pods with the prod label

`kubectl get pods -o wide`	Display Pod IP and node details
`kubectl describe rs nginx-rs`	Display ReplicaSet details
`kubectl get rs nginx-rs -o yaml`	Display ReplicaSet YAML from the cluster

17. Important concepts learned

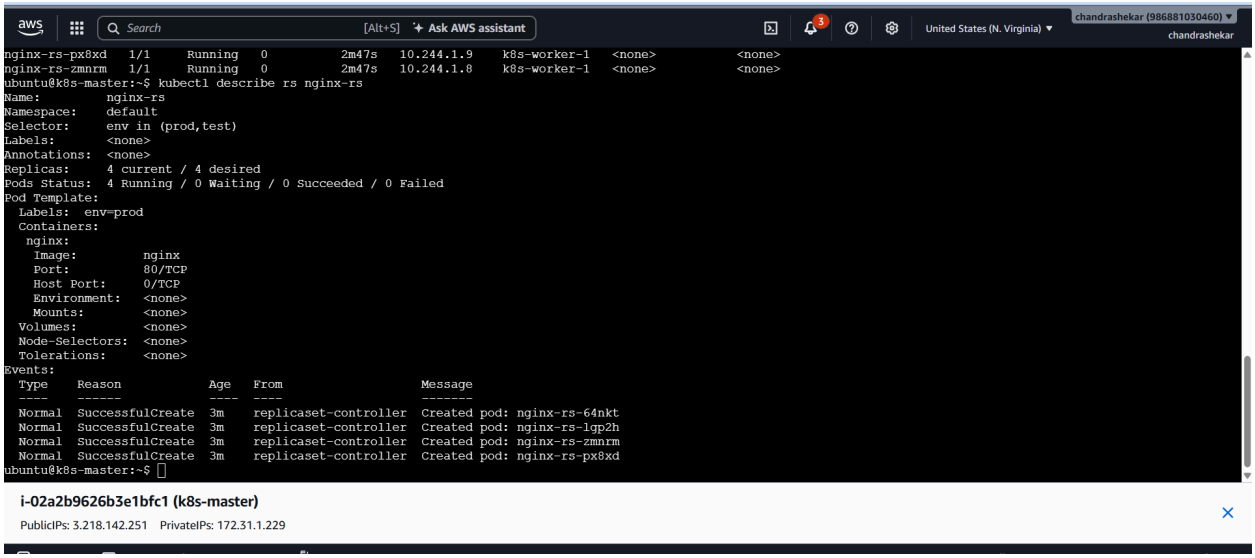
1. ReplicaSet: Maintains the desired number of Pods.
2. Labels: Identify Kubernetes resources using key-value pairs.
3. Selectors: Identify Pods based on labels.
4. matchExpressions: Provides advanced label selection.
5. In operator: Matches any one of the specified values.
6. Pod template: Defines the configuration of Pods created by the ReplicaSet.
7. Self-healing: ReplicaSet creates replacement Pods when managed Pods are deleted or become unavailable.

18. Conclusion

In this task, we successfully created a Kubernetes ReplicaSet named `nginx-rs` using a YAML configuration file.

We configured the ReplicaSet with four replicas and used `matchExpressions` with the `In` operator to match Pods having either `prod` or `test` as their environment label.

Kubernetes successfully created four Nginx Pods with the `env=prod` label. All four Pods reached the Running state and were distributed across the two worker nodes.



6.Deploy a ReplicaSet that excludes pods with the label backend.

K8s-03 – Task 6: Create a ReplicaSet That Excludes Pods with the `backend` Label Using the `NotIn` Operator

Completed successfully

1. Objective

The objective of this task is to create a Kubernetes ReplicaSet that manages Nginx Pods while excluding Pods with the label value `backend`.

In this task, we use:

- ReplicaSet to maintain three Nginx Pods.
- `matchExpressions` for advanced label selection.
- `NotIn` operator to exclude the `backend` label value.
- `tier: frontend` as the label assigned to the created Pods.

2. What is the NotIn operator?

The `NotIn` operator is used in Kubernetes label selectors to exclude specific label values.

For example:

```
None
      matchExpressions:
        - key: tier
          operator: NotIn
          values:
            - backend
```

This means that the selector matches Pods whose `tier` value is not `backend`.

Pod label	Selection result
`tier=frontend`	Matches
`tier=backend`	Excluded
`tier=testing`	Matches
`tier` label absent	Matches

Important: For `NotIn`, Kubernetes also matches Pods that do not have the specified label key.

3. Environment details

Component	Details

Platform	AWS EC2
Operating System	Ubuntu 24.04.5 LTS
Kubernetes version	v1.37.2
Container Runtime	containerd
Master node	k8s-master
Worker node 1	k8s-worker-1
Worker node 2	k8s-worker-2
Resource type	ReplicaSet
ReplicaSet name	nginx-rs-notin
Container image	nginx
Desired replicas	3

4. Verify Kubernetes cluster nodes

Before creating the ReplicaSet, verify that all cluster nodes are ready.

Command:

```
None
kubectl get nodes
```

Output:

```
None
NAME           STATUS   ROLES           AGE   VERSION
k8s-master     Ready    control-plane   4d    v1.37.2
k8s-worker-1   Ready    <none>          4d    v1.37.2
k8s-worker-2   Ready    <none>          4d    v1.37.2
```

Explanation:

All three Kubernetes nodes are in the `Ready` state.

5. Check existing ReplicaSets

Command:

```
None
kubectl get rs
```

Output:

```
None
NAME               DESIRED   CURRENT   READY   AGE
nginx-69b9cdbbdd   2         2         2       3d23h
nginx-rs           4         4         4       9m40s
```

Explanation:

Two existing ReplicaSets were present:

- `nginx-69b9cdbbdd` – created by an earlier Nginx Deployment.
- `nginx-rs` – created during Task 5.

We created a separate ReplicaSet named `nginx-rs-notin` to perform Task 6 without modifying the existing resources.

6. Create the ReplicaSet YAML file

Create a new YAML file using the Nano editor.

Command:

```
None
nano nginx-rs-notin.yaml
```

Add the following configuration:

File name: `nginx-rs-notin.yaml`

```
None
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs-notin
spec:
  replicas: 3
  selector:
    matchExpressions:
      - key: tier
        operator: NotIn
        values:
          - backend
  template:
    metadata:
      labels:
        tier: frontend
```

```
  spec:
    containers:
      - name: nginx
        image: nginx
        ports:
          - containerPort: 80
```

Save the file:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

7. Explanation of YAML configuration

YAML field	Explanation
`apiVersion: apps/v1`	Specifies the Kubernetes API version
`kind: ReplicaSet`	Defines the resource type
`metadata.name`	Name of the ReplicaSet
`replicas: 3`	Maintains three Pods
`selector`	Defines which Pods the ReplicaSet manages

`matchExpressions`	Defines advanced label selection rules
`key: tier`	Label key used for selection
`operator: NotIn`	Excludes the specified label values
`values: backend`	Label value to exclude
`template`	Defines the Pod configuration
`labels.tier: frontend`	Assigns the frontend label to created Pods
`image: nginx`	Uses the Nginx container image
`containerPort: 80`	Specifies the container port

8. Verify the YAML file

Before applying the configuration, display the file contents.

Command:

```
None
cat nginx-rs-notin.yaml
```

Output:

None

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs-notin
spec:
  replicas: 3
  selector:
    matchExpressions:
      - key: tier
        operator: NotIn
        values:
          - backend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Explanation:

The YAML configuration was checked successfully. The indentation and configuration structure were correct.

9. Apply the ReplicaSet configuration

Execute the following command to create the ReplicaSet.

Command:

```
None
      kubectl apply -f nginx-rs-notin.yaml
```

Expected output:

```
None
      replicaset.apps/nginx-rs-notin created
```

Explanation:

The `kubectl apply` command reads the YAML file and creates the ReplicaSet in the Kubernetes cluster.

10. Verify the ReplicaSet

Execute:

Command:

```
None
      kubectl get rs
```

The newly created ReplicaSet should show:

Name	Desired	Current	Ready
nginx-rs-notin	3	3	3

This confirms that Kubernetes created the ReplicaSet and maintained the required three Pods.

11. Verify the ReplicaSet selector

To inspect the ReplicaSet configuration, execute:

Command:

```
None
      kubectl describe rs nginx-rs-notin
```

Important output:

```
None
      Name:         nginx-rs-notin
      Namespace:    default
      Selector:     tier notin (backend)

      Replicas:     3 current / 3 desired

      Pods Status:  3 Running / 0 Waiting / 0 Succeeded / 0 Failed

      Pod Template:
        Labels:  tier=frontend
        Containers:
         nginx:
          Image:         nginx
          Port:          80/TCP
```

Explanation:

- `Selector: tier notin (backend)` confirms that the `NotIn` operator is configured.
- `Replicas: 3 current / 3 desired` confirms that three Pods are maintained.
- `Pods Status: 3 Running` confirms that all three Pods are running.
- `Labels: tier=frontend` confirms the label assigned to the Pods.

12. Verify all three Nginx Pods

To display only the Pods having the `tier=frontend` label, execute:

Command:

```
None
      kubectl get pods -l tier=frontend -o wide
```

Output:

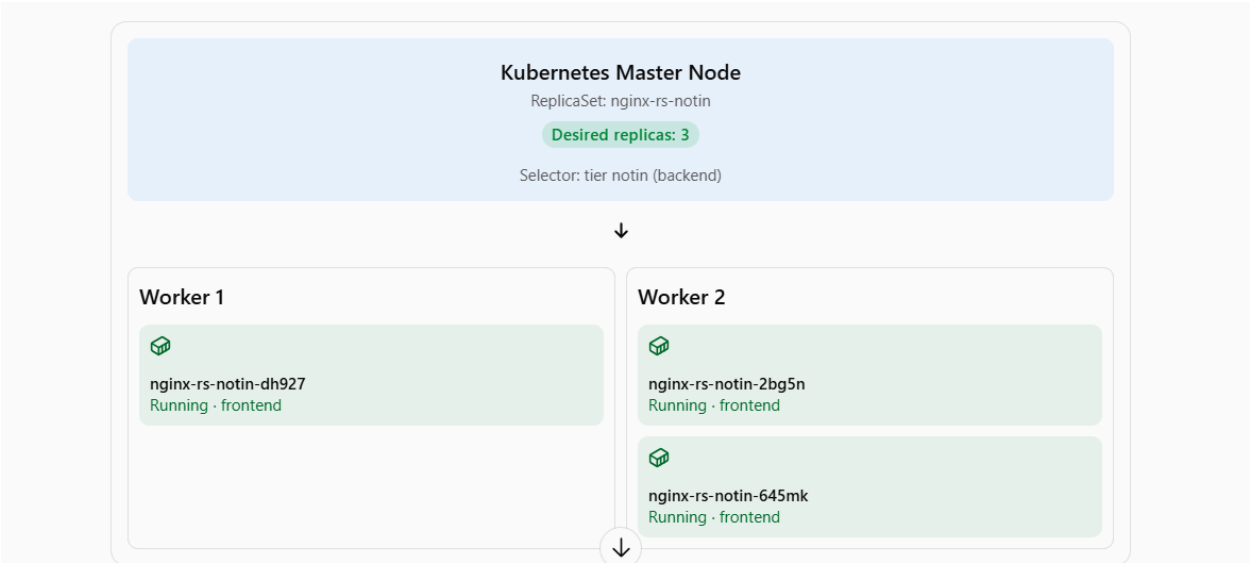
```
None
      NAME                    READY   STATUS    RESTARTS   AGE   IP
      NODE
      nginx-rs-notin-2bg5n    1/1     Running   0          93s
      10.244.2.12   k8s-worker-2
      nginx-rs-notin-645mk    1/1     Running   0          93s
      10.244.2.11   k8s-worker-2
      nginx-rs-notin-dh927    1/1     Running   0          93s
      10.244.1.10   k8s-worker-1
```

Explanation:

- Three Nginx Pods were created successfully.
- All three Pods are in the `Running` state.
- Two Pods are running on `k8s-worker-2`.
- One Pod is running on `k8s-worker-1`.
- Each Pod has a separate IP address.
- All three Pods have the label `tier=frontend`.

13. Kubernetes architecture

The following diagram illustrates the resources created in this task.



14. Important Kubernetes commands

Command	Purpose
`kubectl get nodes`	Display Kubernetes cluster nodes
`kubectl get rs`	List ReplicaSets
`kubectl apply -f nginx-rs-notin.yaml`	Create or update ReplicaSet
`kubectl get pods -l tier=frontend`	Display frontend Pods

`kubectl get pods -o wide`	Display Pod IP and node details
`kubectl describe rs nginx-rs-notin`	Display ReplicaSet details
`kubectl get rs nginx-rs-notin -o yaml`	Display ReplicaSet YAML from the cluster

15. Important concepts learned

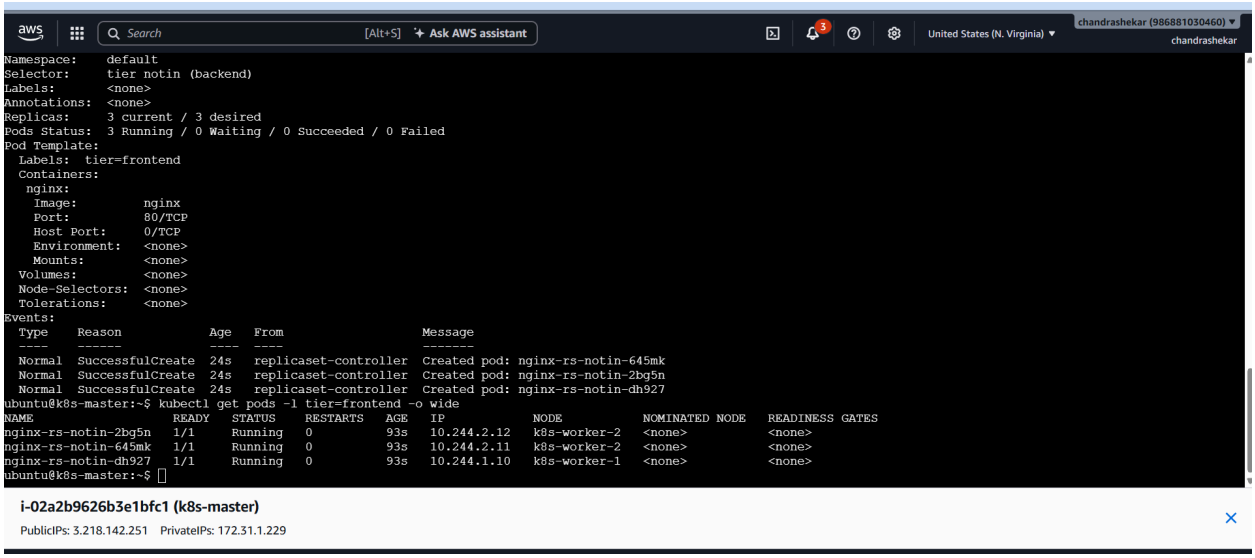
1. ReplicaSet: Maintains the desired number of Pods.
2. Labels: Identify Kubernetes resources using key-value pairs.
3. Selectors: Identify Pods based on labels.
4. matchExpressions: Provides advanced label selection.
5. NotIn operator: Excludes specified label values.
6. Pod template: Defines the configuration of Pods created by the ReplicaSet.
7. Self-healing: ReplicaSet creates replacement Pods when managed Pods are deleted or become unavailable.

16. Conclusion

In this task, we successfully created a Kubernetes ReplicaSet named `nginx-rs-notin` using a YAML configuration file.

We configured the ReplicaSet with three replicas and used `matchExpressions` with the `NotIn` operator to exclude Pods with the label value `backend`.

Kubernetes successfully created three Nginx Pods with the label `tier=frontend`. All three Pods reached the Running state and were distributed across the two worker nodes.



7.Test load balancing across multiple pods using a NodePort service.

K8s-03 – Task 7: Test Load Balancing Across Multiple Pods Using a NodePort Service

1. Objective

The objective of this task is to test Kubernetes Service load balancing across multiple Nginx Pods using a NodePort Service.

In this task, we create a NodePort Service that distributes incoming HTTP requests across four Nginx Pods created in Task 5.

We verify load balancing through:

- Kubernetes ClusterIP.
- Kubernetes NodePort.
- Repeated HTTP requests using curl.
- Different responses from individual Nginx Pods.

2. What is Load Balancing in Kubernetes?

Load balancing means distributing incoming network traffic across multiple backend Pods.

In Kubernetes, a Service provides a stable network endpoint to access a group of Pods.

When multiple Pods match a Service selector, Kubernetes routes traffic to the available endpoints.

Example:

One client sends requests to a Service, and the Service forwards traffic to different Nginx Pods.

3. Environment Details

Component	Details
Platform	AWS EC2
Operating System	Ubuntu 24.04.5 LTS
Kubernetes Version	v1.37.2
Container Runtime	containerd
Master Node	k8s-master
Worker Node 1	k8s-worker-1
Worker Node 2	k8s-worker-2
ReplicaSet	nginx-rs
Number of Nginx Pods	4
Service Name	nginx-lb-nodeport

Service Type	NodePort
NodePort	31009

4. Verify Kubernetes Cluster Nodes

Before creating the NodePort Service, verify that all cluster nodes are ready.

Command:

```
None
kubectl get nodes
```

Output:

```
None
NAME           STATUS   ROLES           AGE   VERSION
k8s-master     Ready    control-plane   4d    v1.37.2
k8s-worker-1   Ready    <none>          4d    v1.37.2
k8s-worker-2   Ready    <none>          4d    v1.37.2
```

Explanation:

All three Kubernetes nodes are in the Ready state.

5. Verify Existing Nginx Pods

We use the four Nginx Pods created in Task 5.

All four Pods have the common label `env=prod`.

Command:

```
None
kubectl get pods -l env=prod -o wide
```

Output:

```
None
NAME             READY   STATUS    RESTARTS   AGE   IP
NODE
nginx-rs-64nkt   1/1     Running   0          22m   10.244.2.9
k8s-worker-2
nginx-rs-lgp2h   1/1     Running   0          22m
10.244.2.10   k8s-worker-2
nginx-rs-px8xd   1/1     Running   0          22m   10.244.1.9
k8s-worker-1
nginx-rs-zmnrm   1/1     Running   0          22m   10.244.1.8
k8s-worker-1
```

Explanation:

- Four Nginx Pods are running successfully.
- All four Pods have the label `env=prod`.
- Two Pods are running on Worker 1.
- Two Pods are running on Worker 2.
- Each Pod has a unique IP address.

6. Check Existing Services

Command:

```
None
kubectl get services
```

Output:

```
None
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP
PORT(S)
kubernetes      ClusterIP   10.96.0.1      <none>
443/TCP
nginx-service   ClusterIP   10.97.163.85   <none>
80/TCP
```

Explanation:

Existing Services were checked before creating a new NodePort Service.

7. Create NodePort Service YAML File

Create a YAML file using the Nano editor.

Command:

```
None
nano nginx-lb-nodeport.yaml
```

Add the following configuration:

File name: `nginx-lb-nodeport.yaml`

```
None
apiVersion: v1
kind: Service
metadata:
  name: nginx-lb-nodeport
spec:
  type: NodePort
  selector:
    env: prod
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
```

Save the file:

1. Press Ctrl + O.
2. Press Enter.
3. Press Ctrl + X.

8. Explanation of YAML Configuration

YAML Field	Explanation
apiVersion: v1	Specifies the Kubernetes API version
kind: Service	Defines the resource as a Service
metadata.name	Name of the Service
type: NodePort	Exposes the Service through a port on each node
selector	Selects Pods using labels
env: prod	Selects the four Nginx Pods from Task 5
port: 80	Service communication port
targetPort: 80	Container port where Nginx listens
protocol: TCP	Uses TCP communication

9. Verify the YAML File

Before applying the configuration, verify the file.

Command:

```
None
cat -n nginx-lb-nodeport.yaml
```

The configuration was verified successfully and contained exactly 12 lines.

10. Apply the NodePort Service

Execute the following command to create the Service.

Command:

```
None
kubectl apply -f nginx-lb-nodeport.yaml
```

Output:

```
None
service/nginx-lb-nodeport created
```

Explanation:

The kubectl apply command reads the YAML file and creates the NodePort Service in the Kubernetes cluster.

11. Verify the NodePort Service

Command:

```
None
kubectl get svc nginx-lb-nodeport
```

Output:

```
None
NAME                TYPE       CLUSTER-IP     EXTERNAL-IP
PORT(S)        AGE
```

```
nginx-lb-nodeport   NodePort   10.97.148.58   <none>
80:31009/TCP   15s
```

Explanation:

The Service was created successfully.

Important details:

Field	Value
Service Name	nginx-lb-nodeport
Type	NodePort
ClusterIP	10.97.148.58
Service Port	80
Node Port	31009
Protocol	TCP

Important concept:

- Port 80 is the Service port.
- TargetPort 80 is the Nginx container port.
- NodePort 31009 is the port exposed on each Kubernetes node.

12. Verify Service Endpoints

To verify which Pods are connected to the Service, execute:

Command:

```
kubectl get endpoints nginx-lb-nodeport
```

Output:

```
NAME                ENDPOINTS
nginx-lb-nodeport   10.244.1.8:80,10.244.1.9:80,10.244.2.10:80
+ 1 more...
```

Explanation:

The Service successfully identified the four backend Nginx Pods.

The older Endpoints API displays a deprecation warning in newer Kubernetes versions. EndpointSlices are the recommended replacement.

13. Describe the NodePort Service

Command:

```
kubectl describe svc nginx-lb-nodeport
```

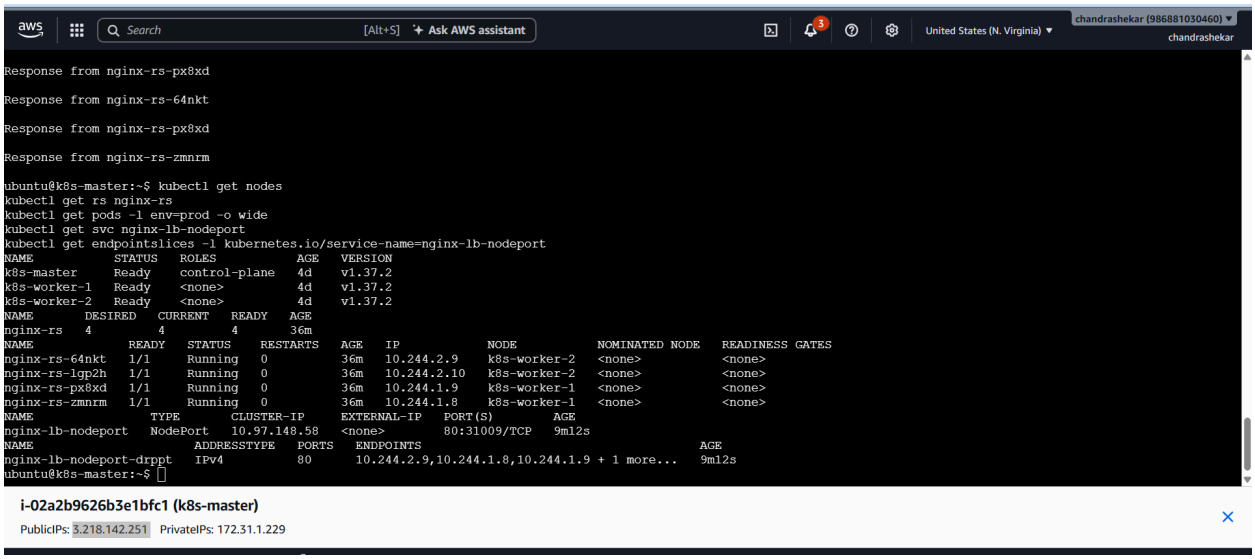
Important output:

```
Name:                     nginx-lb-nodeport
Namespace:                default
Selector:                 env=prod
Type:                     NodePort
IP:                       10.97.148.58
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
```

```
NodePort:                 <unset>  31009/TCP
Endpoints:
10.244.2.9:80,10.244.1.8:80,10.244.1.9:80 + 1 more...
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
```

Explanation:

- Selector `env=prod` ident



8.Delete a ReplicationController without affecting the running pods.

K8s-03 – Task 8: Delete a ReplicationController Without Deleting Its Pods

1. Objective

The objective of this task is to delete a Kubernetes ReplicationController without immediately deleting the Pods managed by it.

Normally, when we delete a ReplicationController, Kubernetes also deletes its dependent Pods.

To prevent this behaviour, Kubernetes provides the `--cascade=orphan` option.

In this task, we use this option to delete the `nginx-rc` ReplicationController while attempting to preserve its five running Nginx Pods.

2. What is a ReplicationController?

A ReplicationController is a Kubernetes resource that ensures a specified number of Pod replicas are running.

For example, if a ReplicationController is configured with five replicas, Kubernetes tries to maintain five Pods.

If a Pod is deleted while the ReplicationController is active, it creates a replacement Pod.

3. What is Orphan Deletion?

Orphan deletion means deleting a controller without automatically deleting its dependent resources.

In Kubernetes, we use:

```
None
kubectl delete rc nginx-rc --cascade=orphan
```

Here:

- `kubectl delete` – deletes a Kubernetes resource.
- `rc` – represents ReplicationController.
- `nginx-rc` – name of the ReplicationController.
- `--cascade=orphan` – prevents automatic cascading deletion of dependent Pods.

The existing Pods become orphaned initially and can potentially be adopted by another matching controller.

4. Environment Details

Component	Details
Platform	AWS EC2
Operating System	Ubuntu 24.04.5 LTS
Kubernetes Version	v1.37.2
Master Node	k8s-master
Worker Node 1	k8s-worker-1
Worker Node 2	k8s-worker-2
ReplicationController	nginx-rc
Initial Replicas	5
Container Image	nginx

5. Verify Kubernetes ReplicationController

Before deleting the ReplicationController, check its current status.

Command:

```
None
kubectl get rc
```

Actual output:

```
None
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   5         5         5       88m
```

Explanation:

- Desired: 5 Pods.
- Current: 5 Pods.
- Ready: 5 Pods.

The ReplicationController successfully maintained five running Nginx Pods.

6. Verify Existing Nginx Pods

Execute the following command to list the Pods managed by the ReplicationController.

Command:

```
None
kubectl get pods -l app=nginx-rc -o wide
```

Actual output before deletion:

```
None
NAME             READY   STATUS    RESTARTS   AGE   IP
NODE
nginx-rc-4gxgc   1/1     Running   0          77m   10.244.1.7
k8s-worker-1
nginx-rc-4hkss   1/1     Running   0          88m   10.244.1.5
k8s-worker-1
nginx-rc-97jc9   1/1     Running   0          77m   10.244.2.8
k8s-worker-2
nginx-rc-lbv6z   1/1     Running   0          88m   10.244.1.6
k8s-worker-1
nginx-rc-rvqmg   1/1     Running   0          86m   10.244.2.7
k8s-worker-2
```

Explanation:

Before deletion, all five Nginx Pods were running successfully.

Three Pods were running on Worker 1, and two Pods were running on Worker 2.

7. Delete ReplicationController Without Cascading Deletion

Execute the following command:

Command:

```
None
kubectl delete rc nginx-rc --cascade=orphan
```

Actual output:

```
None
replicationcontroller "nginx-rc" deleted from default namespace
```

Explanation:

The ReplicationController was successfully deleted.

The `--cascade=orphan` option prevents Kubernetes from immediately deleting its dependent Pods as part of this deletion operation.

8. Verify ReplicationController Deletion

Execute:

```
None
kubectl get rc
```

Actual output:

```
None
No resources found in default namespace.
```

Explanation:

The output confirms that the `nginx-rc` ReplicationController no longer exists.

The command was also executed a second time:

```
None
kubectl delete rc nginx-rc --cascade=orphan
```

Output:

```
None
Error from server (NotFound): replicationcontrollers "nginx-rc" not
found
```

This is expected because the ReplicationController had already been deleted.

9. Verify the Existing Pods After Deletion

Execute:

```
None
kubectl get pods -l app=nginx-rc -o wide
```

Actual output:

```
None
NAME             READY   STATUS    RESTARTS   AGE   IP
NODE
nginx-rc-4gxgc   1/1     Running   0          79m   10.244.1.7
k8s-worker-1
nginx-rc-97jc9   1/1     Running   0          79m   10.244.2.8
k8s-worker-2
nginx-rc-rvqmg   1/1     Running   0          87m   10.244.2.7
k8s-worker-2
```

Observation:

Only three of the original five Pods remained.

At this point, the ReplicationController was deleted successfully, but the expected result of preserving all five Pods was not achieved.

We investigated the reason.

10. Investigate the Missing Pods

To identify what happened, we checked all namespaces for Pods with the `nginx-rc` name.

Command:

```
None
kubectl get pods -A -o wide | grep nginx-rc
```

Actual output:

```
None
default nginx-rc-4gxgc 1/1 Running 0 80m 10.244.1.7 k8s-worker-1
default nginx-rc-97jc9 1/1 Running 0 80m 10.244.2.8 k8s-worker-2
default nginx-rc-rvqmg 1/1 Running 0 88m 10.244.2.7 k8s-worker-2
```

Only three Pods were found.

We then checked Kubernetes events.

Command:

```
None
kubectl get events --sort-by=.lastTimestamp | tail -20
```

Important actual events:

```
None
Normal SuccessfulDelete replicaset/nginx-rs-notin
Deleted pod: nginx-rc-lbv6z

Normal SuccessfulDelete replicaset/nginx-rs-notin
```

```
Deleted pod: nginx-rc-4hkss
```

Explanation:

These events show that the Task 6 ReplicaSet named `nginx-rs-notin` deleted the two missing Pods.

The deletion was not caused directly by the orphan deletion command.

11. Understand the Task 6 ReplicaSet Selector

In Task 6, we created the following ReplicaSet:

File: `nginx-rs-notin.yaml`

```
None
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs-notin
spec:
  replicas: 3
  selector:
    matchExpressions:
      - key: tier
        operator: NotIn
        values:
          - backend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
```

```
        - containerPort: 80
```

The selector uses the `NotIn` operator.

It means that the ReplicaSet selects Pods whose `tier` label is not equal to `backend`.

Important Kubernetes concept:

For a `NotIn` expression, Pods without the specified label key also match the selector.

The five Pods created by `nginx-rc` had the label:

```
None
labels:
  app: nginx-rc
```

They did not have a `tier` label.

Therefore, after the ReplicationController was deleted, the Task 6 ReplicaSet could adopt these orphaned Pods.

The Task 6 ReplicaSet needed only three replicas. After adoption, it reconciled the total number of Pods to its desired count of three by deleting two Pods.

12. Verify ReplicaSet Details

Execute:

```
None
kubectl describe rs nginx-rs-notin
```

Actual important output:

```
None
Name:         nginx-rs-notin
Namespace:    default
Selector:     tier notin (backend)
```

```
Replicas:     3 current / 3 desired

Pods Status:  3 Running / 0 Waiting / 0 Succeeded / 0 Failed
```

Events:

```
None
SuccessfulDelete Deleted pod: nginx-rs-notin-645mk
SuccessfulDelete Deleted pod: nginx-rc-lbv6z
SuccessfulDelete Deleted pod: nginx-rc-4hkss
SuccessfulDelete Deleted pod: nginx-rs-notin-dh927
SuccessfulDelete Deleted pod: nginx-rs-notin-2bg5n
```

This confirms that the Task 6 ReplicaSet reconciled its Pods and deleted the two adopted Nginx Pods.

13. Verify Pod Ownership

To check which controller owns the remaining Pods, execute:

```
None
kubectl get pods -l app=nginx-rc -o yaml | grep -A8 ownerReferences
```

Actual output:

```
None
ownerReferences:
- apiVersion: apps/v1
  blockOwnerDeletion: true
  controller: true
  kind: ReplicaSet
  name: nginx-rs-notin
```

Explanation:

The remaining three Pods are now owned by the `nginx-rs-notin` ReplicaSet.

This confirms that the orphaned Pods were adopted by another matching controller.

14. Verify All ReplicaSets

Execute:

```
None
kubectl get rs
```

Actual output:

```
None
NAME                DESIRED   CURRENT   READY   AGE
nginx-69b9cdbbdd    2         2         2       4d
nginx-rs            4         4         4       68m
nginx-rs-notin      3         3         3       53m
```

Explanation:

The original ReplicationController `nginx-rc` no longer exists.

The other ReplicaSets continue running.

The Task 6 ReplicaSet maintains three running Pods.

15. Important Concepts Learned

Concept	Explanation
ReplicationController	Maintains a desired number of Pod replicas
Cascade deletion	Deletes dependent resources when a controller is deleted
Orphan deletion	Deletes a controller without automatically cascading deletion to its dependents
`--cascade=orphan`	Preserves dependent Pods from immediate cascading deletion
Selector	Identifies Pods using labels and expressions

`NotIn`	Matches values other than the specified values, including missing label keys
Pod adoption	Another matching controller can adopt orphaned Pods
OwnerReferences	Identifies the owning Kubernetes resource
Reconciliation	Controller adjusts resources to match its desired state

16. How to Avoid Selector Overlap

When creating multiple ReplicationControllers or ReplicaSets, use unique and carefully designed labels.

For example, use a dedicated label:

```
None
metadata:
  labels:
    workload: nginx-rc
    tier: frontend
```

And use a selector that specifically identifies the intended workload.

This helps prevent one controller from unintentionally adopting Pods belonging to another workload.

17. Conclusion

In this task, we successfully deleted the `nginx-rc` ReplicationController using:

```
None
kubectl delete rc nginx-rc --cascade=orphan
```

The ReplicationController was removed successfully.

However, only three of the original five Pods remained running. Investigation showed that the Task 6 ReplicaSet `nginx-rs-notin` adopted the orphaned Pods because its `NotIn` selector also matched Pods without a `tier` label.

The ReplicaSet then reconciled its desired replica count of three and deleted two Pods.

This practical demonstrated orphan deletion, Pod adoption, label selectors, owner references, and Kubernetes controller reconciliation.

Task 8 – ReplicationController deletion using orphan propagation: Command successfully executed; the selector-overlap behaviour was identified and verified.

18. Screenshots to Include in the Assignment

Capture these screenshots:

1. `kubectl get rc` – initial ReplicationController with five replicas.
2. `kubectl get pods -l app=nginx-rc -o wide` – initial five running Pods.
3. `kubectl delete rc nginx-rc --cascade=orphan` – successful deletion.
4. `kubectl get rc` – no ReplicationController remaining.
5. `kubectl get pods -l app=nginx-rc -o wide` – three remaining Pods.
6. `kubectl get events --sort-by=.lastTimestamp | tail -20` – events showing deletion of two Pods by the Task 6 ReplicaSet.
7. `kubectl describe rs nginx-rs-notin` – selector and deletion events.
8. `kubectl get pods -l app=nginx-rc -o yaml | grep -A8 ownerReferences` – new owner reference.
9. `kubectl get rs` – final ReplicaSet status.

